test-coverage

Test coverage ledger — Challenges submodule (meta Challenge-of-Challenges)

Revision	Created	Last modified	Status
1	2026-05-19	2026-05-19	active

Round-304 deep-doc + Challenge enrichment for the cross-cutting Challenge bank. This document is the inventory ledger consumed by `challenges_describe_challenge.sh` (the meta-runner that walks each bank and asserts each has a runner + paired-mutation evidence path).

Cascaded mandate (verbatim 2026-05-19, per CONST-049 §11.4.17):

> "all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!"

Table of contents

1. Scope and role

The `challenges/` submodule is **the** cross-cutting Challenge bank for the HelixCode family. Every consumer (Panoptic, security, helix_qa, helix_llm, the core HelixCode app, plus any external consumer that imports `digital.vasic.challenges`) loads Challenge definitions from the banks shipped here, executes them via `pkg/runner`, and reports via `pkg/report`. Because this submodule is **itself** test infrastructure, its anti-bluff posture must be **meta**: a Challenge bank that claims "PASS" on a broken-for-end-user feature is the canonical failure mode CONST-035 / Article XI §11.9 / round-304 was created to close. The meta-runner (`challenges_describe_challenge.sh`) is the gate.

2. Bank inventory — `banks/examples/`

28 generic JSON banks under `banks/examples/`. Each bank ships a set of challenge definitions (id + name + description + assertions) consumable by `pkg/bank`. Banks asserted present by the describe-runner:

File	Purpose	Loader
`concurrency.json`	Concurrent challenge dispatch	`pkg/bank`
`concurrency-safety-validation.json`	Race / data-race coverage	`pkg/bank`
`cross-platform-build.json`	Per-OS build sweep	`pkg/bank`
`documentation-completeness-challenges.json`	Doc-completeness gates	`pkg/bank`
`e2e-userflow.json`	E2E user-flow bank (userflow runner)	`pkg/userflow`
`format-detection.json`	Format-sniff coverage	`pkg/bank`
`format-edge-cases-challenges.json`	Format edge-cases	`pkg/bank`
`format-parsing.json`	Format parse correctness	`pkg/bank`
`lazy-loading.json`	Lazy-load behaviour	`pkg/bank`
`memory.json`	Memory-pressure gates	`pkg/bank`
`monitoring.json`	Monitoring exposure	`pkg/bank`
`monitoring-metrics-challenges.json`	Prometheus metrics shape	`pkg/metrics`
`network-protocols.json`	Network-protocol coverage	`pkg/bank`
`performance.json`		`pkg/bank`

File	Purpose	Loader
	Performance SLO checks	
`performance-optimization-validation.json`	Perf-opt verification	`pkg/bank`
`platform-coverage-challenges.json`	Per-platform coverage	`pkg/userflow`
`protocol-resilience-challenges.json`	Protocol resilience	`pkg/bank`
`resilience.json`	Resilience baseline	`pkg/bank`
`security-challenges.json`	Security challenge bank	`pkg/bank`
`security.json`	Security baseline	`pkg/bank`
`security-scanning-validation.json`	Security-scan gates	`pkg/bank`
`stress-responsiveness-challenges.json`	Stress responsiveness	`pkg/bank`
`test-coverage.json`	Test-coverage gates	`pkg/bank`
`timeout-recovery-challenges.json`	Timeout / recovery	`pkg/runner`
`ui-accessibility.json`	UI a11y bank	`pkg/userflow`
`ui-automation-android.json`	Android UI bank	`pkg/userflow`
`ui-automation-desktop.json`	Desktop UI bank	`pkg/userflow`
`ui-automation-web.json`	Web UI bank	`pkg/userflow`

The describe-runner asserts every file in the table above exists and parses (`jq` or equivalent), with a planted-rename paired-mutation proving the inventory check itself is not a bluff.

3. Bank inventory — `banks/yole/`

YAML feature-coverage banks plus fixtures (consumer-side Yole IDE example wiring):

File	Purpose
`coverage-matrix.md`	Feature × variant × evidence matrix
`file-browser-save-functionality.yaml`	File-browser save E2E
`version-consistency-validation.yaml`	

File	Purpose
	Version-consistency check
`feature-coverage/feature-1-syntax-highlighting.yaml`	Syntax-highlighting bank
`feature-coverage/feature-2-source-code-support.yaml`	Source-code support bank
`feature-coverage/feature-3-autocomplete.yaml`	Autocomplete bank
`feature-coverage/feature-4a-lsp-completion.yaml`	LSP completion bank
`feature-coverage/feature-4b-diagnostics-hover-gotodef.yaml`	LSP diagnostics / hover / goto-def bank
`feature-coverage/feature-4c-refactoring.yaml`	Refactoring bank
`feature-coverage/feature-5-import.yaml`	Import bank
`fixtures/hello-world.kt`, `hello-world.rs`, `rename-target.rs`, `sample-class.py`, `type-error.rs`, `test-import.docx`	Source-language + binary fixtures

The describe-runner asserts the directory is present and contains at least 7 feature-coverage YAML files.

4. Shell-script Challenges — `challenges/scripts/`

16 shell-script challenges + 1 baseline reference text. Each script must be executable, parseable under `sh -n` (per CONST §11.4.67), and respond to `--help` or return a recognisable signature when invoked.

Script	Anti-bluff role
`anchor_manifest_challenge.sh`	Anchor manifest integrity
`android_save_challenge.sh`	Android file-save Challenge
`bluff_scanner_challenge.sh`	Static bluff-pattern scanner
`challenges_compile_challenge.sh`	Go module compile gate
`challenges_functionality_challenge.sh`	Functional gate
`challenges_unit_challenge.sh`	Unit-test gate
`chaos_failure_injection_challenge.sh`	Chaos-injection Challenge
`ddos_health_flood_challenge.sh`	DDoS / flood resilience

Script	Anti-bluff role
host_no_auto_suspend_challenge.sh	Host power-management ban (CONST-033)
mutation_ratchet_challenge.sh	Mutation-score ratchet
no_suspend_calls_challenge.sh	Source-tree no-suspend scan
recording_pipeline_challenge.sh	Recording-pipeline integrity
scaling_horizontal_challenge.sh	Horizontal-scale resilience
stress_sustained_load_challenge.sh	Sustained-load Challenge
ui_terminal_interaction_challenge.sh	TUI interaction Challenge
ux_end_to_end_flow_challenge.sh	Full UX flow Challenge
baselines/bluff-baseline.txt	Bluff-scanner baseline reference

5. Production-code primitives exercised

The describe-runner cross-references the production-code surface listed in ARCHITECTURE.md against the bank inventory above:

Package	Public surface exercised by banks
pkg/challenge	Challenge, BaseChallenge, Config, Result, ProgressReporter, StatusStuck
pkg/registry	Registry, dependency ordering (Kahn)
pkg/runner	Runner, RunAll, RunSequence, RunParallel, liveness monitor
pkg/assertion	Engine, 16 built-in evaluators
pkg/report	Markdown / JSON / HTML reporters
pkg/bank	JSON / YAML bank loader (consumes banks/examples/*, banks/yole/*)
pkg/monitor	EventCollector + WebSocket dashboard
pkg/metrics	Prometheus metrics emitter
pkg/plugin	Plugin registry
pkg/infra	Containers bridge
pkg/userflow	8 adapter interfaces × 21 impls × 19 challenge templates × 12 evaluators
cmd/userflow-runner	CLI runner for userflow Challenges

Package	Public surface exercised by banks
`lib/anti_bluff.sh`	Anti-bluff helper library for shell-script Challenges

A bank that references a primitive missing from the table above OR a primitive in the table not exercised by any bank is a CONST-048 coverage gap — promoted to an Issues.md entry per §11.4.15 + §11.4.16.

6. Locale fixtures

`challenges/fixtures/payloads.json` ships 5-locale coverage so the bank loader + assertion engine survive non-ASCII bytes end-to-end:

Locale	Code	Script
English	`en`	Latin
German	`de`	Latin + umlaut (`ä ö ü ß`)
Spanish	`es`	Latin + diacritic (`á é í ñ`)
Japanese	`ja`	CJK (kanji + hiragana + katakana)
Serbian	`sr`	Cyrillic

Mandated by CONST-046 (no hardcoded user-facing strings) + Round-304 parity with rounds 220 / 295 / 298 / 300.

7. Paired-mutation invariants

Per CONST-035 / §1.1 / Article XI §11.9 every gate ships with a paired mutation. The describe-runner's `--anti-bluff-mutate` flag plants a deliberate inventory mismatch in a TMP COPY of this ledger (renames the `NoopTranslator`-equivalent `concurrency.json` token, for example) and asserts the gate FAILS with exit code 99. The original tree is never mutated. A meta-runner that PASSes its own mutation is itself a bluff and a CONST-035 violation.

Mutation matrix:

Mutation	Target	Expected outcome
Rename bank file token in ledger	`docs/test-coverage.md` (tmp copy)	gate FAIL, exit 99
Strip operator mandate verbatim	`docs/test-coverage.md` (tmp copy)	gate FAIL, exit 99
Strip `round-304` marker	`README.md` (tmp copy)	

Mutation	Target	Expected outcome
		gate FAIL, exit 99

8. Anti-bluff cross-checks

The describe-runner is intentionally **redundant** with the per-bank challenges in `challenges/scripts/`. Redundancy is the point: if any single per-bank Challenge silently regresses to a bluff (PASS on broken feature), the meta-runner catches it via inventory cross-check. The two-layer guard is the closure of the recursion that motivated round-304.